

Smart Kumbh Mela — Agent & Build Reference

1. Build Order & Dependencies

- Data models + DB connection — every collection (users, familyMembers, zones, scanLogs, crowdLogs, cameraSources, lostPersonCases, emergencies, volunteerReports). Nothing else works without this.
- Auth + role guard (JWT) — visitor / volunteer / admin / superadmin. Required before any role-scoped route.
- Zones + admin/volunteer hierarchy — zone CRUD, admin-only volunteer creation. Required before SOS routing, crowd alert fan-out, and volunteer assignment.
- Visitor + family registration + digital QR generation.
- QR scan + passphrase reveal + scan logging.
- Crowd monitoring pipeline — AI service (YOLOv8) + WebSocket bridge + Socket.IO broadcast. Independent of registration; can be built in parallel with steps 4–5.
- Lost-person face matching — depends on registration (photo optional/fallback) and the AI service/WebSocket bridge from step 6.
- SOS/emergency — depends on zones + role hierarchy (step 3) for type-based routing.
- Navigation — depends on crowd monitoring data (step 6) and zones.facilities.
- Volunteer dashboard (task list, notifications) — depends on auth (step 2) and SOS/case data (steps 7–8) to have something to show.
- Phase 2: Twilio SMS/WhatsApp officer alerts, analytics view, misconduct reporting workflow — layer on top once MVP is stable.

2. Folder / Repo Structure

```
kumbh-mela-safety-platform/  
  client/                                React app (single app, role-based views)  
    src/  
      views/  
        visitor/  
        volunteer/  
        admin/  
        superadmin/  
      components/  
      services/                          API + Socket.IO clients  
      hooks/  
      routes/  
  
  server/                                Node.js + Express + Socket.IO  
    src/  
      models/                            Mongoose schemas — one file per collection  
      routes/                            REST routes, grouped by feature  
      controllers/  
      middleware/                        auth.js, roleGuard.js  
      sockets/                           Socket.IO event handlers  
      services/  
        aiServiceClient.js               WebSocket client -> AI service  
        twilioClient.js                  SMS + Voice  
    config/
```

```

ai-service/
  app/
    main.py
    websocket_server.py
    crowd_detection.py
    face_matcher.py
    camera_source.py
    models/
      Python + FastAPI
      YOLOv8 pipeline, frame sampler
      face-matching pipeline
      webcam / CCTV-RTSP / recorded-file switch
      YOLOv8 pretrained weights

docs/
  Kumbh_Mela_Architecture_Reference.docx
  Kumbh_Mela_Agent_Build_Reference.docx (this file)

.env.example
README.md

```

3. Consolidated API Reference

Method	Endpoint	Purpose
POST	/api/auth/register	Visitor self-registration
POST	/api/family-members	Add family member + generate digital QR
POST	/api/family-members/:id/issue-band	Mark physical wristband issued (checkpoint)
GET	/api/scan/:qrCode	Start a scan — requires volunteer/admin JWT
POST	/api/scan/:qrCode/verify	Verify passphrase, reveal profile, log scan
POST	/api/lost-person-cases	Raise a lost-person case (photo optional)
PATCH	/api/lost-person-cases/:id	Confirm found — stops face matching
POST	/api/sos	Raise SOS (medical / police / mass-incident)
PATCH	/api/sos/:id/assign	Assign SOS to a responder
GET	/api/zones/status	Current density status per zone
POST	/api/volunteers	Admin-only — create volunteer account
GET	/api/volunteers/tasks	Volunteer's assigned tasks
POST	/api/volunteers/:id/report	Public misconduct report by volunteer ID (Phase 2)

4. Consolidated Real-Time Events

Event	Direction	Payload / Notes
crowd:update	AI service → Node backend (WebSocket)	{zoneId, peopleCount, densityLevel, ts}
start_matching	Node backend → AI service (WebSocket)	{caseId, referenceImage, zones}
stop_matching	Node backend → AI service (WebSocket)	{caseId}
match_found	AI service → Node backend (WebSocket)	{caseId, zoneId, confidence, ts}
zone:status	Node backend → all clients (Socket.IO)	live per-zone density, broadcast, no auth
alert:zone	Node backend → admins (Socket.IO)	zone crossed high-density threshold
sos:new	Node backend → assigned admin (Socket.IO)	new SOS, paired with Twilio voice call + SMS

5. Data Collections — Quick Field Reference

Collection	Key Fields
users	role, name, phone, passwordHash, zoneId (volunteer/admin)
familyMembers	registeredBy, name, age, guardianContact, address, photoUrl (optional), digitalQR, physicalBandIssued
zones	name, boundary, zoneAdminId, facilities[] (medical camp / exit / help desk)
cameraSources	zoneId, type (webcam/cctv/recorded), sourceUrl
scanLogs	familyMemberId, scannedBy, location, timestamp
crowdLogs	zoneId, peopleCount, densityLevel, timestamp
lostPersonCases	familyMemberId, raisedBy, referenceImage, status (open/found), matchedZone, matchedAt
emergencies	raisedBy, type (medical/police/mass-incident), zoneId, status, assignedTo
volunteerReports	volunteerId, reportedBy, reason, status (open/resolved/warned/deactivated)

6. Functional Requirements Checklist

6.1 Registration + QR

- Visitor can register self + family members from anywhere (no checkpoint required)
- Digital QR generated per family member immediately on registration
- Photo is optional at registration
- Checkpoint volunteer can convert digital QR to physical wristband
- QR scan redirects to a passphrase-gated reveal page — no direct info exposure
- Passphrase is auto-shown in the scanning volunteer's/admin's own logged-in session
- Every scan writes a scanLogs entry: who, where, when, whose QR

6.2 Crowd Monitoring

- Camera source configurable per zone: webcam, CCTV/RTSP, or recorded file — swappable without code change
- YOLOv8 computes people count + density (low/medium/high) per zone
- crowd:update pushed over WebSocket, stored in crowdLogs
- zone:status broadcast to all clients live, no auth/permissions required
- High-density zone triggers alert:zone to admins
- (Phase 2) SMS/WhatsApp to zone + government officers on high density
- (Phase 2) Historical crowdLogs feed an analytics/planning view

6.3 Lost Person Detection

- Face matching only runs after a case is explicitly raised (not continuous)
- Reference image = uploaded photo, falling back to registered photo if present
- Match alerts nearest admin/volunteer to that zone + the case raiser
- Match is a lead only — case stays open until manually confirmed
- Case auto-expires (stop_matching) after a configurable timeout if unconfirmed
- QR-based identification remains a separate, independent path from face-matching cases

6.4 SOS / Emergency

- Medical and Police/Security are separate SOS types
- User-raised SOS routes to the zone's type-specific admin, not directly to police/medical
- SOS triggers a Twilio voice call + SMS, not just an in-app alert
- Volunteer-raised mass-incident SOS skips normal routing — goes to zone admin + super admin, high-priority
- SOS queues locally on poor/no connectivity and auto-retries on reconnect

6.5 Navigation

- Zone status view shows current density per zone (color-coded), not turn-by-turn routing
- Alternative-zone suggestion via simple distance/adjacency lookup
- Facility routing (medical camps/exits/help desks) uses real map/route guidance (Leaflet + OSRM)

6.6 Volunteer Management

- Only admin/super admin can create volunteer accounts — no self-registration
- Volunteer account includes: task list, zone view, notifications, emergency-report shortcut
- (Phase 2) Any visitor can report a volunteer by ID; admin can resolve/warn/deactivate

6.7 Roles & Access

- Four roles enforced end-to-end: visitor, volunteer, admin, superadmin
- Admin dashboard scoped to their own zone
- Super admin dashboard scoped to all zones